Assignment 8

This exercise sheet is dedicated to piecewise polynomial interpolation, apart from Exercise 29 which refers to (global) polynomial interpolation. Note that it might be helpful to solve Exercise 30, at least question 1, before tackling the graded exercise.

Graded exercise

Exercise 28 (Piecewise linear interpolation)

In class we have seen an error estimate for the piecewise linear Lagrange interpolant (that is, first order spline) $S_1 \in \mathcal{S}_1$ interpolating a function $f \in C^2([a,b])$ on a grid $\{x_i\}_{i=0}^m$ with nodes $a = x_0 < x_1 < \cdots < x_m = b$, where $h = \max_{i=1,\dots,m}(x_i - x_{i-1})$. In this exercise, we illustrate this estimate numerically. For learning purposes, to solve this exercise you are **not** allowed to use `scipy` routines. Beyond this exercise, though, be aware that piecewise linear as well as cubic spline interpolation are implemented in the `scipy` library `scipy.interpolate`.

(a) Write a PYTHON function `plotspline` that, taking in input a function handle to the function f to be interpolated and an array `x` containing the points $\{x_i\}_{i=0}^m$ of the grid, plots the function f and its piecewise linear interpolant on the grid and returns (an estimate for) the error $\|f - S_1\|_\infty$ $(= \|f - S_1\|_{C^0([x_0,x_m])})$.

Hint: To plot and compute the error, use a loop over the intervals. On each interval $[x_{i-1}, x_i]$, $i = 1, \dots, m$, you can approximate the maximum norm error by computing the maximum of the difference between the two functions on a uniform grid with 20 points in $[x_{i-1}, x_i]$ (extrema included). The overall maximum norm error is obtained by taking the largest error over the intervals.

(b) Write a PYTHON script that tests your routine from **(a)** with the function $f(x) = 1/(1 + 25x^2)$ on $[-1, 1]$ and equispaced grid points $x_i = -1 + \frac{1}{2}i$, $i = 0, \dots, 4$, and plots the function f and its linear interpolant on the same figure. Save the obtained plot and hand it in (together with the script).

(c) Write a PYTHON script that, for $f(x) = 1/(1+25x^2)$ and for $m = 2^k$ with $k = 1, \ldots, 10$, computes the maximum norm error of the piecewise linear interpolant using equispaced points $x_i = -1 + \frac{2}{m}i$, $i = 0, \ldots, m$. Use the routine from task **(a)** to compute the error for each value of m. The script must also produce a double logarithmic (loglog) plot of the error versus the number m of subintervals. How do you expect the curve to look like, according to the approximation result seen in class? Use the `numpy` functions `polyfit` or `numpy.polynomial.Polynomial.fit` to verify, with your script, that the convergence rate of the error equals (approximately) the theoretical one. Remember to hand in also the plot, an explanation of the expected rate and the result that you get via fitting.

Other exercises

Exercise 29 (Noisy data)

Suppose we have data f_i, corresponding to locations $x_i \in [a,b]$, $i = 0, \ldots, m$, which are obtained from measurements with a relative error bounded by ε in absolute value. This means that, instead of the exact values f_i, we have to work with perturbed values $\widetilde{f}_i = (1+\varepsilon_i)\, f_i$ with $|\varepsilon_i| \leq \varepsilon$. This leads to a perturbed interpolant $\widetilde{P}(x)$, with $\widetilde{P}(x_i) = \widetilde{f}_i$ for $i = 0, 1, \ldots, m$. Here we consider global polynomial interpolation, that is, $\tilde{P} \in \mathbb{P}_m([a,b])$. Show that, if f is the function underlying the measurements (without noise), which we assume to be smooth, then, for every $x \in [a,b]$,

$$\left| f(x) - \widetilde{P}(x) \right| \leq \frac{1}{(m+1)!} \max_{\xi \in [a,b]} |f^{(m+1)}(\xi)| \cdot |\omega(x)| + \varepsilon \max_{0 \leq i \leq m} |f_i| \cdot \lambda_m(x),$$

where $\omega(x)$ is the monic polynomial associated to the nodes and $\lambda_m(x) = \sum_{i=0}^{m} |L_i(x)|$ is the Lebesgue function. The latter is related to the Lebesgue constant by $\Lambda_m = \max_{x \in [a,b]} \lambda_m(x)$.

Remark: Because of the presence of the Lebesgue function, the estimate above shows another advantage of Chebyshev versus equidistant nodes.

Exercise 30 (Spline computation)

We are given the points $\{x_0, x_1, x_2\}$ and the data

x_i	0	1	2
f_i	0	t	0

where $t \in \mathbb{R}$. Compute (by hand) the following interpolants in dependence of the real parameter t:

1. the linear spline interpolant $S_1 \in \mathcal{S}_1$ (note that this corresponds to the piecewise linear Lagrange interpolant),
2. the cubic spline interpolant $S_3 \in \mathcal{S}_3$ with natural boundary conditions, namely, $S_3''(x_0) = S_3''(x_2) = 0$.

Use PYTHON to draw both spline interpolants for $t = 2$.